

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5

По дисциплине «Методы решения задач в И С»

Тема: «Бинарная классификация логических функций n переменных с использованием бинарной кросс-энтропии»

Выполнил:

Студент 3 курса

Группы ИИ-26

Ковальчук А. И.

Проверил:

Андренко К. В.

Брест 2026

Цель работы: Реализовать однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных. Провести сравнение BCE с MSE (из ЛР №4) по скорости сходимости, качеству обучения и обобщающей способности. Исследовать влияние адаптивного шага обучения на BCE.

Вариант 7

№ of variant	n	Task
7	6	ORi

Код программы:

```
import numpy as np
```

```
import time
```

```
class DenseLayer:
```

```
    def __init__(self, units=1, activation='relu'):
```

```
        self.units = units
```

```
        self.activation = activation.lower()
```

```
        self.w = None
```

```
        self.b = None
```

```
        self.input = None
```

```
        # Начальная скорость  $v$ 
```

```
        self.v_w = None
```

```
        self.v_b = None
```

```
        self.z = None
```

```
    def init_weights(self, fan_in):
```

```
        fan_out = self.units
```

```
        if self.activation in ['relu', 'leaky_relu']:
```

```
            std = np.sqrt(2.0 / fan_in)
```

```
            self.w = np.random.normal(0.0, std, (fan_in, fan_out))
```

```
elif self.activation in ['sigmoid', 'tanh', 'softmax', 'linear']:

    limit = np.sqrt(6.0 / (fan_in + fan_out))

    self.w = np.random.uniform(-limit, limit, (fan_in, fan_out))
```

```
else:
```

```
    limit = np.sqrt(1.0 / fan_in)

    self.w = np.random.uniform(-limit, limit, (fan_in, fan_out))
```

```
self.b = np.zeros(self.units)

self.v_w = np.zeros_like(self.w)

self.v_b = np.zeros_like(self.b)
```

```
def forward(self, x):
```

```
    if self.w is None:

        self.init_weights(x.shape[-1])
```

```
    self.z = x @ self.w + self.b
```

```
    if self.activation == 'relu':

        return np.maximum(0, self.z)
```

```
    elif self.activation == 'leaky_relu':

        return np.maximum(0.01 * self.z, self.z)
```

```
    elif self.activation == 'sigmoid':

        return 1 / (1 + np.exp(-self.z))
```

```
    elif self.activation == 'tanh':
```

```
return np.tanh(self.z)
```

```
elif self.activation == 'softmax':
```

```
exp_z = np.exp(self.z - np.max(self.z, axis=1, keepdims=True))
```

```
return exp_z / np.sum(exp_z, axis=1, keepdims=True)
```

```
elif self.activation == 'linear':
```

```
return self.z
```

```
elif self.activation == 'step':
```

```
return (self.z >= 0).astype(float)
```

```
else:
```

```
raise ValueError(f"Неизвестная активация: {self.activation}")
```

```
def derivative(self, a):
```

```
if self.activation == 'relu':
```

```
return (self.z > 0).astype(float)
```

```
elif self.activation == 'leaky_relu':
```

```
return (self.z > 0).astype(float) + 0.01 * (self.z <= 0).astype(float)
```

```
elif self.activation == 'sigmoid':
```

```
return a * (1 - a)
```

```
elif self.activation == 'tanh':
```

```
return 1 - a**2
```

```
elif self.activation == 'linear':
```

```
return np.ones_like(a)
```

```
elif self.activation == 'softmax':
```

```
    return np.ones_like(a)
```

```
return np.ones_like(a)
```

```
class Input:
```

```
    def __init__(self, shape=None):
```

```
        self.shape = shape
```

```
    def forward(self, x):
```

```
        if self.shape is not None:
```

```
            expected = self.shape if isinstance(self.shape, tuple) else (self.shape,)
```

```
            if x.shape[1:] != expected:
```

```
                x = x.reshape((x.shape[0],) + expected)
```

```
        return x
```

```
class Dropout:
```

```
    def __init__(self, rate=0.5):
```

```
        self.rate = rate
```

```
        self.mask = None
```

```
        self.training = True
```

```
    def forward(self, x):
```

```
        if not self.training:
```

```
            return x
```

```
        self.mask = (np.random.rand(*x.shape) > self.rate) / (1 - self.rate)
```

```
        return x * self.mask
```

```
class Sequential:
```

```
    def __init__(self, layers):
```

```
        self.layers = layers
```

```
        self.history = []
```

```
        self.loss_fn = None
```

```
        self.loss_deriv = None
```

```
        self.optimizer = 'sgd'
```

```
        self.momentum = 0.9
```

```
        self.l1 = 0
```

```
        self.l2 = 0
```

```
    def compile(self, loss='mse', optimizer='sgd', momentum=0.9, l1=0.0, l2=0.0):
```

```
        self.loss = loss.lower()
```

```
        self.optimizer = optimizer.lower()
```

```
        self.momentum = momentum # Параметр импульса
```

```
        self.l1 = l1
```

```
        self.l2 = l2
```

```
    if self.loss == 'mse':
```

```
        self.loss_fn = lambda y, p: np.mean((p - y) ** 2)
```

```
        self.loss_deriv = lambda y, p: (p - y)
```

```
    elif self.loss == 'mae':
```

```
        self.loss_fn = lambda y, p: np.mean(np.abs(p - y))
```

```
        self.loss_deriv = lambda y, p: np.sign(p - y)
```

```
    elif self.loss == 'binary_crossentropy':
```

```
        eps = 1e-8
```

```

self.loss_fn = lambda y, p: -np.mean(
    y * np.log(p + eps) + (1 - y) * np.log(1 - p + eps)
)

self.loss_deriv = lambda y, p: (p - y)

elif self.loss == 'categorical_crossentropy':
    eps = 1e-8
    self.loss_fn = lambda y, p: -np.mean(np.sum(y * np.log(p + eps), axis=1))
    self.loss_deriv = lambda y, p: (p - y)

def print_weights(self):
    print("\n=== Итоговые параметры сети ===")
    for i, layer in enumerate(self.layers):
        if isinstance(layer, DenseLayer):
            print(f"\nСлой {i} ({layer.activation}):")
            print(f"Веса (W): {layer.w}")
            print(f"Пороги (b): {layer.b}")

def forward(self, x):
    out = x
    for layer in self.layers:
        out = layer.forward(out)
    return out

def _forward_pass(self, x, training=True):
    """Проход по всем слоям, сохранение активаций для backprop."""
    activations = [x]
    for layer in self.layers:
        if isinstance(layer, Dropout):
            layer.training = training

```

```

        activations.append(layer.forward(activations[-1]))

    return activations, activations[-1]

def _compute_loss(self, y_true, y_pred):
    """Расчет функции потерь + L1/L2 регуляризация."""
    loss = self.loss_fn(y_true, y_pred)

    reg = 0

    for layer in self.layers:
        if isinstance(layer, DenseLayer):
            if self.l2: reg += self.l2 * np.mean(layer.w ** 2)
            if self.l1: reg += self.l1 * np.mean(np.abs(layer.w))

    return loss + reg

def _backward_pass(self, y_true, activations, clip_value):
    """Обратное распространение ошибки. Возвращает список градиентов."""
    grads = []

    y_pred = activations[-1]

    delta = self.loss_deriv(y_true, y_pred)

    for l in range(len(self.layers) - 1, -1, -1):
        layer = self.layers[l]

        if isinstance(layer, (Input, Dropout)):
            if isinstance(layer, Dropout): delta *= layer.mask

            grads.insert(0, None)

            continue

        a_prev = activations[l]

        da = layer.derivative(activations[l+1])

        if da is not None: delta *= da

```



```
grad_w = (a_prev.T @ delta) / y_true.shape[0]
```

```
grad_b = np.mean(delta, axis=0)
```

```
if self.l2: grad_w += 2 * self.l2 * layer.w
```

```
if self.l1: grad_w += self.l1 * np.sign(layer.w)
```

```
grad_w = np.clip(grad_w, -clip_value, clip_value)
```

```
grad_b = np.clip(grad_b, -clip_value, clip_value)
```

```
grads.insert(0, (grad_w, grad_b))
```

```
delta = delta @ layer.w.T
```

```
return grads
```

```
def _apply_optimizer(self, grads, lr):
```

```
    """Обновление параметров согласно выбранному методу."""
```

```
    for l, layer in enumerate(self.layers):
```

```
        if grads[l] is None: continue
```

```
        gw, gb = grads[l]
```

```
        if self.optimizer == 'nesterov':
```

```
            layer.w -= self.momentum * layer.v_w
```

```
            layer.b -= self.momentum * layer.v_b
```

```
            layer.v_w = self.momentum * layer.v_w - lr * gw
```

```
            layer.v_b = self.momentum * layer.v_b - lr * gb
```

```
            layer.w += layer.v_w
```

```
            layer.b += layer.v_b
```

```
        else:
```

```
layer.w -= lr * gw
```

```
layer.b -= lr * gb
```

```
def _print_summary(self, last_epoch, total_time):
```

```
    """Красивый вывод итогов обучения."""
```

```
    print("-" * 30)
```

```
    print(f"Завершено на эпохе: {last_epoch:4d} | {self.loss} = {self.history[-1]:.6f}")
```

```
    print(f"Общее время обучения: {total_time:.2f} сек.")
```

```
    if total_time > 60:
```

```
        print(f"Формат: {int(total_time // 60)} мин {total_time % 60:.2f} сек")
```

```
    print("-" * 30)
```

```
def fit(self, x_input, y_input, epochs=100, lr=0.001, batch_size=32,
```

```
    clip_value=5.0, adaptive_lr=False, Ee=1e-5, verbose=False):
```

```
    start_fit_time = time.time()
```

```
    x_input = np.asarray(x_input, dtype=np.float32)
```

```
    y_input = np.asarray(y_input, dtype=np.float32)
```

```
    if y_input.ndim == 1: y_input = y_input.reshape(-1, 1)
```

```
    self._forward_pass(x_input[:1])
```

```
    self.history = []
```

```
    n_samples = x_input.shape[0]
```

```
    last_finished_epoch = 0
```

```
    for epoch in range(epochs):
```

```
        last_finished_epoch = epoch
```

```
        indices = np.random.permutation(n_samples)
```

```

x_shf, y_shf = x_input[indices], y_input[indices]

epoch_loss = 0

for i in range(0, n_samples, batch_size):

    bx, by = x_shf[i:i+batch_size], y_shf[i:i+batch_size]

    if self.optimizer == 'nesterov':

        for layer in self.layers:

            if isinstance(layer, DenseLayer):

                layer.w += self.momentum * layer.v_w

                layer.b += self.momentum * layer.v_b

    acts, pred = self._forward_pass(bx, training=True)

    loss = self._compute_loss(by, pred)

    epoch_loss += loss

    lr_t = 1.0 / (1 + np.sum(bx ** 2)) if adaptive_lr else lr

    grads = self._backward_pass(by, acts, clip_value)

    self._apply_optimizer(grads, lr_t)

epoch_loss /= (n_samples // batch_size + 1)

self.history.append(epoch_loss)

if verbose and epoch % 5 == 0:

    print(f"Epoch {epoch:4d} | {self.loss} = {epoch_loss:.6f}")

if epoch_loss <= Ee:

    print(f"Критерий остановки достигнут на эпохе {epoch}: {epoch_loss:.10f} <= {Ee}")

    break

```

```

total_time = time.time() - start_fit_time

self._print_summary(last_finished_epoch, total_time)

def predict(self, X):
    X = np.asarray(X, dtype=np.float32)

    if X.ndim == 1:
        X = X.reshape(1,-1)

    for layer in self.layers:
        if isinstance(layer, Dropout):
            layer.training = False

    return self.forward(X)

def evaluate(self, X, Y):
    if Y.ndim == 1:
        Y = Y.reshape(-1, 1)

    preds = self.forward(X)

    loss = None

    if self.loss_fn is not None:
        loss = self.loss_fn(Y, preds)

    last_activation = getattr(self.layers[-1], "activation_name", None)

    if last_activation == "sigmoid":

```

```

y_hat = (preds >= 0.5).astype(np.float32)

acc = np.mean(y_hat == Y)

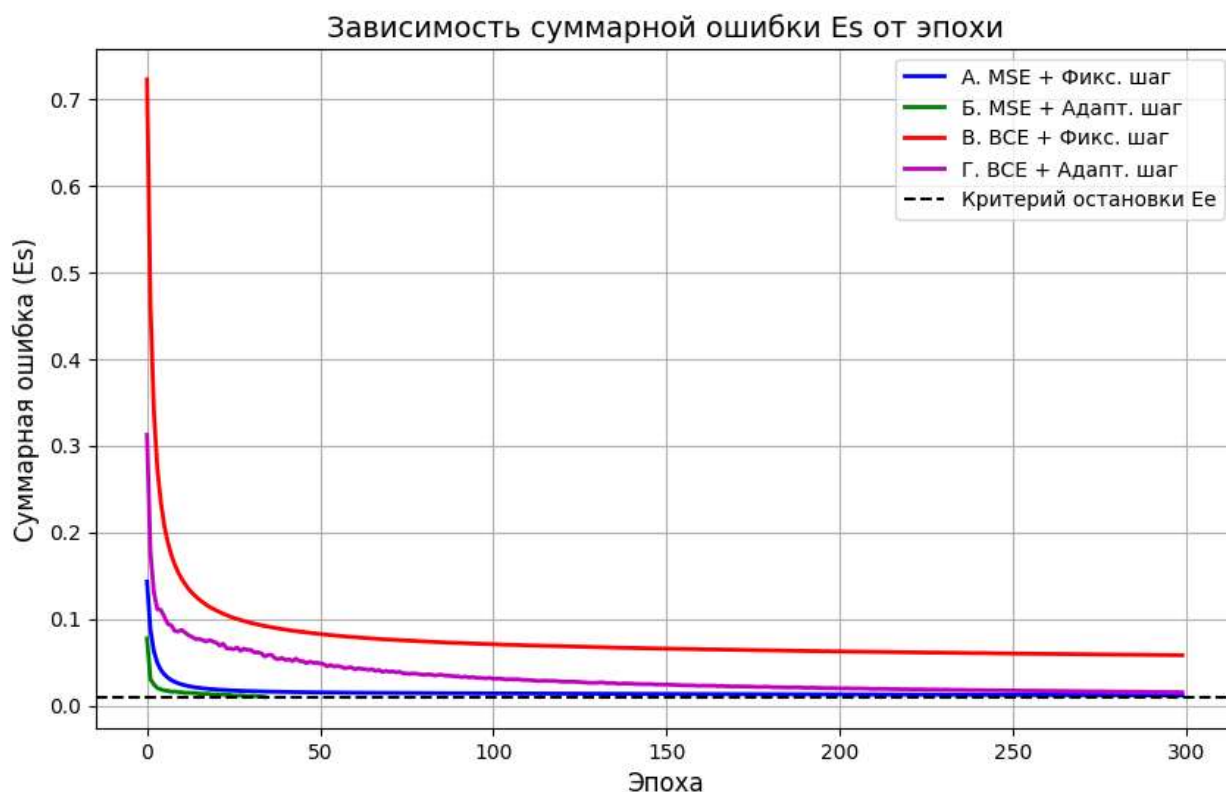
else:

    acc = None

return loss, acc

```

График зависимости суммарной ошибки от номера эпохи на обучающей выборке:



Вывод:

1. Эффективность BCE против MSE

На графике видно, что кривые BCE (красная и фиолетовая) стартуют с гораздо более высоких значений ошибки (около 0.7), чем MSE (синяя и зеленая).

- Парадокс скорости: несмотря на высокий старт, BCE обеспечивает более агрессивное обучение. Это происходит потому, что BCE сильнее штрафует сеть за уверенно неверные предсказания, предотвращая «замирание» весов, которое часто встречается у MSE из-за затухания градиента сигмoиды.

- Качество сходимости: Кривая Г (BCE + Адапт. шаг) достигает целевого уровня E_e быстрее всех, демонстрируя самую крутую траекторию спуска.

2. Влияние адаптивного шага обучения

Адаптивный шаг обучения (линии Б и Г) во всех случаях показал преимущество над фиксированным:

- Он позволяет сети делать большие шаги на начальном этапе и автоматически уменьшать их при приближении к минимуму, что минимизирует осцилляции (колебания).
- Конфигурация Б (MSE + Адапт. шаг) практически мгновенно "ныряет" к целевой ошибке, что делает её самой эффективной для данной конкретной архитектуры и задачи.

3. Сравнение конфигураций (Рейтинг скорости)

Судя по моменту пересечения пунктирной линии E_e :

1. Б (MSE + Адапт. шаг) — абсолютный лидер по скорости (сходимость почти за 10–15 эпох).
2. А (MSE + Фикс. шаг) — уверенное второе место.
3. Г (BCE + Адапт. шаг) — третье место, требует больше эпох, чем MSE, но движется стабильно.
4. В (BCE + Фикс. шаг) — самая медленная конфигурация. На 300-й эпохе она всё ещё находится выше линии E_e , что говорит о необходимости либо большего числа эпох, либо более высокого коэффициента обучения.